

----- **1º Ato Chat Global**

- Instalar Reverb
- Rodar Reverb
- Configurar o Docker
- Reiniciar o Docker
- Criar/Configurar o componente chat_global.js
- Importar o chat_global.js em app.js
- Criar um Canal privado

----- **2º Ato Lista de Presença(Usuarios Logados)**

- Modificar o routes/channels.php
- Criar/Configurar o componente resources/js/presence_list.js
- Importar o present_list.js em app.js

----- **3º Ato Chat Privado entre 2 pessoas**

- Rota em web.php
- ChatController metodo chat
- ChatController metodo store
- Criar/Configurar o componente resources/js/chat.js
- Importar o chat.js em app.js
- Criar/Configurar o evento NewMessage
- Criar um canal privado em channels.php

----- **4º Ato Enviar notificação**

- Criar/Configurar o componente resources/js/notification.js
- Refatorar o controller ChatController
- Criar/Configurar a notification NewMessageNotification
- Importar o notification.js em app.js

- Instalação do reverb -> laravel.com/docs/11.x/reverb#main-content

php artisan install:broadcasting

- Rodar o reverb

php artisan reverb:start

- Configurar o docker porta 8080

docker-compose.yml

laravel.test:

```
...
ports:
  ...
  - "8080:8080"
```

OBS: Depois que fazer essa modificação: sail up -d

- Frontend = chat-global.blade.php

<x-app-layout>

<div class="flex flex-col justify-end h-full w-full" x-data="chat_global({{ auth()->user() }})">

<div class="flex flex-col flex-1 gap-y-2 pt-8 pr-4 overflow-y-auto" x-ref="messageContainer">

<template x-for="(message, index) in messages" :key="index">

<div :class="{ 'flex flex-col items-end w-full': message.userid == user.id, 'flex flex-col items-start w-full':

message.from_user_id != user.id}">

<p class="font-bold text-xs mb-1" x-text="message.username"></p>

<p :class="{ 'rounded-full px-4 py-2 bg-blue-100': message.userid == user.id, 'rounded-full px-4 py-2 bg-gray-100':

message.userid != user.id}" x-text="message.message"></p>

</div>

</template>

</div>

<form @submit.prevent="sendMessage">

@csrf

<div class="mt-2 flex rounded-md shadow-sm">

<div class="relative flex flex-grow items-stretch focus-within:z-10">

<input

type="text"

class="block w-full rounded-none rounded-l-md border-0 py-1.5 text-gray-900 ring-1 ring-inset ring-gray-300

placeholder:text-gray-400 focus:ring-2 focus:ring-inset focus:ring-indigo-600 sm:text-sm sm:leading-6"

placeholder="Aoba"

name="text"

x-model="message"

>

</div>

<button type="submit"

class="relative -ml-px inline-flex items-center gap-x-1.5 rounded-r-md px-3 py-2 text-sm font-semibold text-gray-900

ring-1 ring-inset ring-gray-300 hover:bg-gray-50">

enviar

</button>

</div>

</form>

</div>

</x-app-layout>

- Criar/Configurar o componente chat_global.js

export default (user) => ({

user: user,

messages: [],

message: "",

channel: null,

//Sempre que o componente alpine e inicializado e roda essa função , aqui

fazer a conexão com o websocket

init() {

this.channel=Echo.private('chat.globalroom').listenForWhisper('message',(message)=>{ this.messages.push(message)})

},

sendMessage() {

let message = {message: this.message,username: this.user.name,userid: this.user.id}

this.messages.push(message);

this.message = "";

this.channel.whisper('message', message);

}

})

- Importar o chat_global.js em app.js ,

```
import ChatGlobal from './chat_global.js'
```

```
Alpine.data('chat_global', ChatGlobal);
```

- Criar um Canal privado

```
routes/channels.php
```

```
Broadcast::channel('chat.globalroom', function($user){  
    return true;  
});
```

===== Lista de Presença(Usuarios Logados)

OBS: frontend sidebar.blade.php

- Modificar o routes/channels.php

```
Broadcast::channel('chat.globalroom', function($user){  
    return ['id' => $user->id, 'name' => $user->name];  
});
```

- Criar/Configurar o componente resources/js/presence_list.js

```
export default (user_id) => ({  
    user_id: user_id,  
    users: [],  
    init() {  
        Echo.join('chat.globalroom')  
            .here( (users) => {  
                this.users = users;  
            })  
            .joining( (user) => {  
                this.users.push(user)  
            })  
            .leaving((user) => {  
                this.users = this.users.filter((userInPresenceList) => userInPresenceList.id !== user.id)  
            })  
            .error((error) => {  
                console.error(error);  
            })  
    }  
})
```

- Importar o present_list.js em app.js

```
import PresenceList from './presence_list.js'
```

```
Alpine.data('presence_list', PresenceList);
```

===== Chat Privado entre 2 pessoas

web.php

```
Route::get('/user/{user}', [\App\Http\Controllers\ChatController::class, 'chat'])
```

```
->middleware(['auth'])
```

```
->name('chat.user');
```

ChatController.php

```
public function chat(User $user)  
{  
    $chat = Chat::with('messages')->whereHas('participants', function ($query) {  
        $query->where('user_id', auth()->id());  
    })->whereHas('participants', function ($query) use ($user) {  
        $query->where('user_id', $user->id);  
    })->first();  
  
    if (!$chat) {  
        $chat = Chat::create();  
        ChatUser::create(['chat_id' => $chat->id, 'user_id' => auth()->id()]);  
        ChatUser::create(['chat_id' => $chat->id, 'user_id' => $user->id]);  
    }  
  
    $chat->messages = $chat->messages->sortBy('created_at');  
    return view('chat', ['user' => $user, 'chat' => $chat]);  
}
```

Frontend(chat.blade.php)

```
<x-app-layout>
<div class="flex flex-col justify-end h-full w-full" x-data="chat({{auth()->id()}}, {{ $user->id }}, {{ $chat }})">
  <div class="flex flex-row w-full justify-center font-bold text-md text-gray-400">
    <p class="underline">{{ $user->name }}</p>
  </div>
  <div class="flex flex-col flex-1 gap-y-2 pt-8 pr-4 overflow-y-auto" x-ref="messageContainer">
    <template x-for="message in chat.messages" :key="message.id">
      <div :class="{ 'flex flex-row justify-end w-full': message.user_id == logged_user_id, 'flex flex-row justify-start w-full':
message.user_id != logged_user_id}">
        <p :class="{ 'rounded-full px-4 py-2 bg-blue-100': message.user_id == logged_user_id, 'rounded-full px-4 py-2 bg-gray-
100': message.user_id != logged_user_id}" x-text="message.content"></p>
      </div>
    </template>
    <template x-if="chat_user_typing">
      <div class="mt-4 flex flex-row">
        <p>{{ $user->name }} está digitando</p>
        <div class="flex flex-row gap-x-1">
          <div class="ml-1 animate-bounce"></div>
          <span class="animate-[bounce_1.5s_infinite]"></span>
          <span class="animate-bounce"></span>
        </div>
      </div>
    </template>
  </div>
  <form action="{{ route('chat.store', $chat->id) }}" method="POST" @submit="channel.whisper('stopTyping', {})">
    @csrf
    <div class="mt-2 flex rounded-md shadow-sm">
      <div class="relative flex flex-grow items-stretch focus-within:z-10">
        <input
          type="text"
          @input="sendTyping"
          class="block w-full rounded-none rounded-l-md border-0 py-1.5 text-gray-900 ring-1 ring-inset ring-gray-300
placeholder:text-gray-400 focus:ring-2 focus:ring-inset focus:ring-indigo-600 sm:text-sm sm:leading-6"
          placeholder="Aoba"
          name="content"
        >
      </div>
      <button type="submit"
        class="relative -ml-px inline-flex items-center gap-x-1.5 rounded-r-md px-3 py-2 text-sm font-semibold text-gray-900
ring-1 ring-inset ring-gray-300 hover:bg-gray-50">
        enviar
      </button>
    </div>
  </form>
</div>
</x-app-layout>
```

ChatController.php

```
public function store(Request $request, Chat $chat)
{
    $message = $chat->messages()->create([
        'user_id' => auth()->id(),
        'content' => $request->get('content')
    ]);
    // $user = $chat->participants()->whereNot('users.id', auth()->id())->first();
    // $user->notify(new NewMessageNotification(auth()->user(), $chat, $message));
    NewMessage::dispatch(auth()->id(), $chat, $message);
    return redirect()->back();
}
```

- Criar/Configurar o componente resources/js/chat.js

```
export default (logged_user_id, chat_user_id, chat) => ({
  logged_user_id: logged_user_id,
  chat_user_id: chat_user_id,
  chat: chat,
  chat_user_typing: false,
  channel: null,
  typingTimer: null,
  init() {
    this.channel = Echo.private(`chat.${this.chat.id}`)
      .listen('NewMessage', (e) => {
        if (e.message.user_id !== this.logged_user_id) {
          this.chat.messages.push(e.message)
          this.$nextTick(() => this.scrollToBottom())
        }
      })
      .listenForWhisper('typing', (e) => {
        this.chat_user_typing = true;
        this.$nextTick(() => this.scrollToBottom())
      })
      .listenForWhisper('stopTyping', (e) => {
        this.chat_user_typing = false;
        this.$nextTick(() => this.scrollToBottom())
      })

    this.$nextTick(() => this.scrollToBottom())
  },
  sendTyping() {
    if (this.typingTimer) {
      clearTimeout(this.typingTimer)
    }
    this.channel
      .whisper('typing', {})
      .then(() => {
        this.typingTimer = setTimeout(() => {
          this.channel.whisper('stopTyping', {})
        }, 1000)
      })
  },
  scrollToBottom() {
    const container = this.$refs.messageContainer
    container.scrollTop = container.scrollHeight
  }
})
```

- Importar o chat.js em app.js

```
import Chat from './chat.js'
Alpine.data('chat', Chat);
```

- Criar/Configurar o evento NewMessage

CMD: sail art make:event NewMessage

<?php

```
namespace App\Events;
use App\Models\Chat;
use App\Models\Message;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;
class NewMessage implements ShouldBroadcast {
    use Dispatchable, InteractsWithSockets, SerializesModels;
    public function __construct(protected string $from_user_id, public Chat $chat, public Message $message) {}
    public function broadcastOn(): array {
        return [new PrivateChannel('chat.' . $this->chat->id),];
    }
}
```

- Criar um canal privado em channels.php

```
Broadcast::channel('chat.{chat_id}', function($user, $chat_id) {
    return $user->chats()->where('chat_id', $chat_id)->first();
});

=====Enviar notificação=====

Frontend = notification.blade.php
<div x-data="notification({{ auth()->id() }})" aria-live="assertive" class="pointer-events-none fixed inset-0 flex items-end px-4 py-6 sm:items-start sm:p-6">
    <div class="flex w-full flex-col items-center space-y-4 sm:items-end">
        <template x-for="notification in notifications" :key="notification.message.id">
            <div
                x-data="open = true"
                x-init="setTimeout(() => {open = false; notifications = notifications.filter(not => not.message.id != notification.message.id)}, 10000)"
                x-show="open"
                x-transition:enter="transform ease-out duration-300 transition"
                x-transition:enter-start="translate-y-2 opacity-0 sm:translate-y-0 sm:translate-x-2"
                x-transition:enter-end="translate-y-0 opacity-100 sm:translate-x-0"
                x-transition:leave="transition ease-in duration-100"
                x-transition:leave-start="opacity-100"
                x-transition:leave-end="opacity-0"
                class="pointer-events-auto flex w-full max-w-md rounded-lg bg-white shadow-lg ring-1 ring-black ring-opacity-5">
                <div class="w-0 flex-1 p-4">
                    <div class="flex items-start">
                        <div class="flex-shrink-0 pt-0.5">
                            
                        </div>
                        <div class="ml-3 w-0 flex-1">
                            <p class="text-sm font-medium text-gray-900" x-text="notification.user.name"></p>
                            <p class="mt-1 text-sm text-gray-500" x-text="notification.message.content"></p>
                        </div>
                    </div>
                    <div class="flex border-l border-gray-200">
                        <a
                            :href="route('chat.user', notification.user.id)"
                            class="flex w-full items-center justify-center rounded-none rounded-r-lg border border-transparent p-4 text-sm font-medium text-indigo-600 hover:text-indigo-500 focus:outline-none focus:ring-2 focus:ring-indigo-500">Responder</a>
                    </div>
                </div>
            </template>
        </div>
    </div>
```

- Criar/Configurar o componente resources/js/notification.js

```
export default (user_id) => ({
    user_id: user_id,
    open: false,
    notifications: [],
    init() {
        Echo.private(`App.Models.User.${this.user_id}`)
            .notification((notification) => {
                if (!route().current('chat.user', { user: notification.user.id })) {
                    this.notifications.push(notification)
                }
            })
    }
})
```

- ChatController descomentar

```
$user = $chat->participants()->whereNot('users.id', auth()->id()->first());
$user->notify(new NewMessageNotification(auth()->user(), $chat, $message));
```

- Criar/Configurar a notification NewMessageNotification

```
CMD: sail art make:notification NewMessageNotification
```

```
- app/Notifications/NewMessageNotification.php
```

```

<?php
namespace App\Notifications;
use App\Models\Chat;
use App\Models\Message;
use App\Models\User;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Messages\BroadcastMessage;
use Illuminate\Notifications\Messages\MailMessage;
use Illuminate\Notifications\Notification;
class NewMessageNotification extends Notification
{
    use Queueable;
    public function __construct(public User $user, public Chat $chat, public Message $message)
    {}
    public function via(object $notifiable): array
    {
        return ['broadcast'];
    }
    public function toBroadcast(object $notifiable): BroadcastMessage
    {
        return new BroadcastMessage([
            'user' => $this->user,
            'chat' => $this->chat,
            'message' => $this->message,
        ]);
    }
}
- Importar o notification.js em app.js
import Notification from './notification.js'
Alpine.data('chat',Notification);

```

-- Ate o minuto 02:21:42 daqui pra frente deploy(AWS) da aplicação